

Algorithm for file updates in Python

Project description

The task is to regularly update a file with a list of employees who have access to restricted content. I will create an algorithm to check if certain IP addresses are on a remove list and, if so, remove those employees from the file that contains employees who have access.

Open the file that contains the allow list

To open the file that contained the list with allowed employees, I used the following.

```
In [2]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

Read the file contents

To read the contents of the file I used `.read()` to change the `allow_list` file to a string I wrote

```
ip_addresses = file.read()
```

```
In [4]: # Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

Which produced the following

```
ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9
192.168.52.90
192.168.158.170
192.168.90.124
192.168.186.176
192.168.133.188
192.168.203.198
192.168.201.40
192.168.218.219
192.168.52.37
192.168.156.224
192.168.60.153
192.168.58.57
192.168.69.116
```

Convert the string into a list

The IP addresses need to be in a list format to remove certain employees from the list so I updated the variable `ip_addresses`

```
ip_addresses = ip_addresses.split()
```

The following was the result.

```
['ip_address', '192.168.25.60', '192.168.205.12', '192.168.97.225', '192.168.6.9', '192.168.52.90', '192.168.158.170', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.201.40', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.58.57', '192.168.69.116']
```

Iterate through the remove list

The remove list contains all of the ip addresses of the users that should be removed from the list containing employees with permissions. To do this I built an iterative statement that iterates through the remove list using `element` as the loop variable.

```
# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Display `element` in every iteration

    print(element)
```

Update the file with the revised list of IP addresses

From there I wrote a conditional statement within the iterative that checked if the loop variable element was present in the list of employees with access and, if so, removed them from that list using.

```
if element in remove_list:

    # then current element should be removed from `ip_addresses`

    ip_addresses.remove(element)
```

`.remove()` will delete the first occurrence of a specified value from a list but since this list has no duplicates this method was ok to use.

The IP addresses were removed, and the file needed to be updated with the new list. To do this, the `ip_address` file needed to be converted back into a string.

```
ip_addresses = "\n".join(ip_addresses)
```

`"\n"` was used to instruct python to place each element on a new line.

To write over the file that was assigned to the `import_file` variable, I wrote a `with` statement that would update the file.

```
with open(ip_addresses, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

Summary

An algorithm was created to remove IP addresses that were specified in `remove_list` from the file that contained the IP addresses of the employees who were given access to restricted content `"allow_list.txt"`. It involved opening the file, converting it to a string so it could be read, and then converted into a list so that it could be used in an iterative statement. It iterated through each IP address in `remove_list` and checked if it was in the `"allow_list.txt"` (which was stored as the variable `ip_addresses`) and if it was it used `.remove()` to remove it from that list. After this was completed, `.join()` was used to convert the ip addresses from the allow list

(ip_addresses) back into a string so it could then be overwritten as an updated "allow_list.txt" with the updated IP addresses.